

Experiment No.: 07

Title: Integrate Kubernetes and Docker

Batch: B-1

Roll No.: 16010422234

Experiment No.: 07

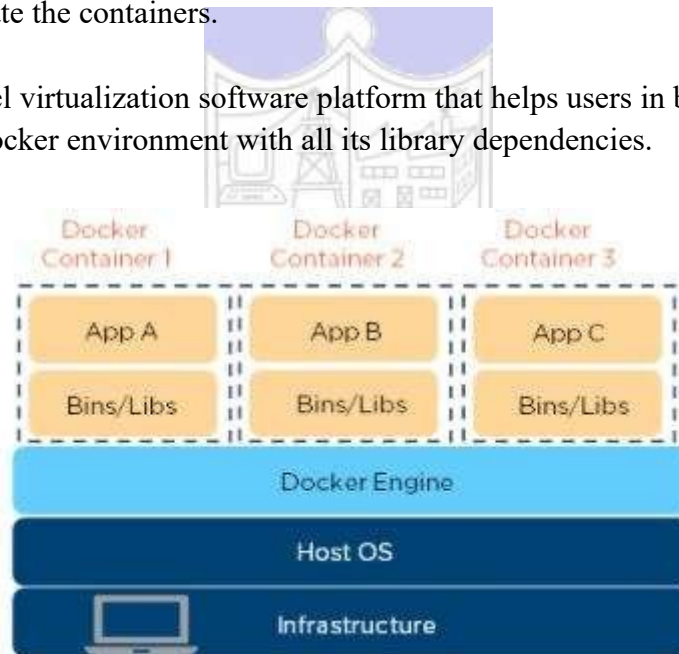
Aim: To create Docker image using Dockerfile for NodeJS application, run and Push it on DockerHub.

Resources needed: Docker Hub Account, Kubernetes

Theory:

Kubernetes and Docker are both popular technologies for managing containers, but they are used for different purposes. Kubernetes is an orchestration platform that provides a higher-level abstraction for managing containers, while Docker is a containerization technology that provides a lower-level runtime for containers. To integrate Kubernetes and Docker, you need to use Docker to build and package your application as a container image, and then use Kubernetes to manage and orchestrate the containers.

Docker is an OS-level virtualization software platform that helps users in building and managing applications in the Docker environment with all its library dependencies.



Docker image is a read-only, inert template that comes with instructions. In Docker, everything basically revolves around images. An image consists of a collection of files (or layers) that pack together all the necessities—such as dependencies, source code, and libraries—needed to set up a completely functional container environment. Images are stored on a Docker registry, such as the Docker Hub, or on a local registry.

Docker Container is a lightweight software package that includes all the dependencies (frameworks, libraries, etc.) required to execute an application. The running image is the container.

Dockerfile is a simple text file that consists of instructions to build Docker images. Dockerfile is used to create Docker images. Dockerfile is nothing but a blueprint for the image. All the Docker images have Dockerfile. Dockerfile consists of specific commands that guide you on how to build a specific Docker image.

The specific commands you can use in a dockerfile are: FROM, PULL, RUN, and CMD

FROM - Creates a layer from the ubuntu:18.04

PULL - Adds files from your Docker repository

RUN - Builds your container

CMD - Specifies what command to run within the container

Create a Docker image using two methods:

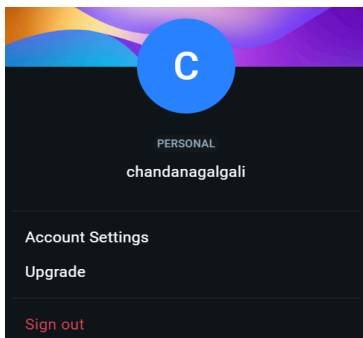
1. Develop all the code locally(on your local computer). And package all the code as Docker image using Dockerfile.
2. Run Docker container. Then place all your code inside the Docker container(Enter into docker container using *docker exec* command). And build an image using the running Docker container.

The below diagram is a **Dockerfile example** for the NodeJS application:

Dockerfile Command Explanation	Dockerfile code
1. Install Node version 13	FROM node:13-alpine
2. Create folder inside Docker Image	RUN mkdir -p /home/app
3. Copy NodeJS app files to Image folder	COPY ./erp /home/app
4. Set as Working Directory	WORKDIR /home/app
5. Running the App Server	CMD ["node", "server.js"]

Procedure:

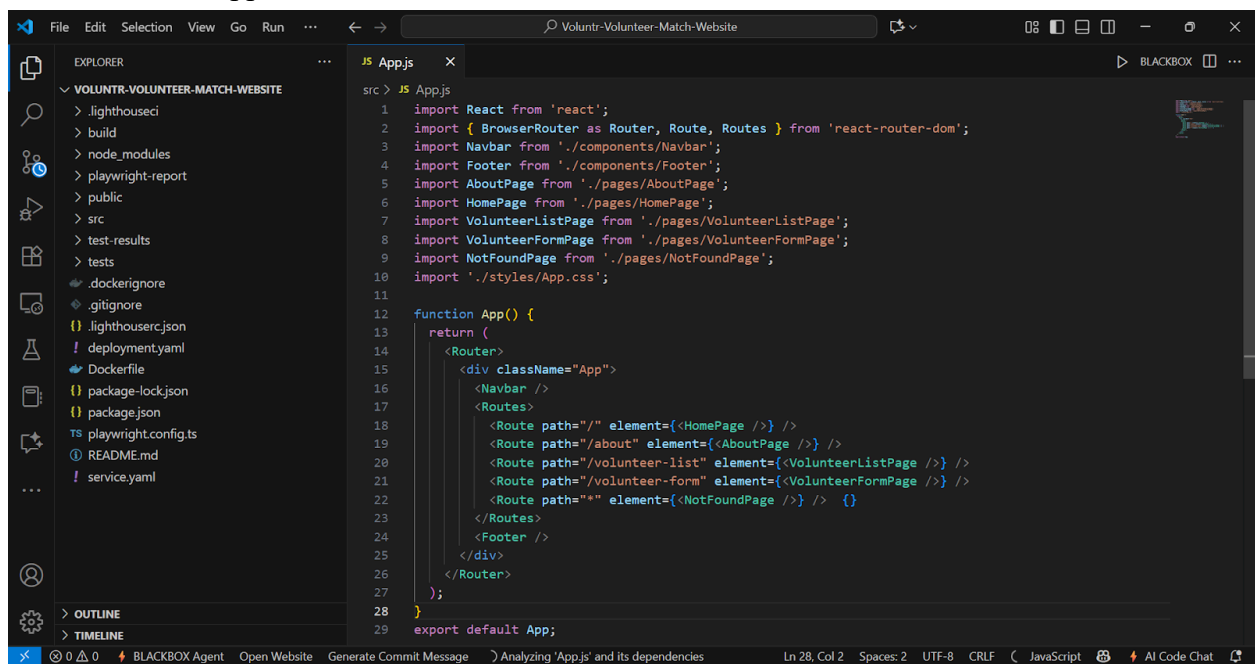
1. Create an Account on Docker Hub.



2. Install Docker and explore Docker commands.
3. Write a NodeJS Application.
4. Write a Dockerfile for NodeJS Application.
5. Create/build an image and check for its availability in the local computer.
6. Run the created image.
7. Push the image on Docker Hub.
8. Use Kubernetes to deploy the Docker image to a cluster
 - a. Create a deployment
 - b. Create a service
9. Modify the code, create/build and run the updated image

Results: (Document with screenshots)

1. NodeJS Application code



```

src > JS Appjs
1  import React from 'react';
2  import { BrowserRouter as Router, Route, Routes } from 'react-router-dom';
3  import Navbar from './components/Navbar';
4  import Footer from './components/Footer';
5  import AboutPage from './pages/AboutPage';
6  import HomePage from './pages/HomePage';
7  import VolunteerListPage from './pages/VolunteerListPage';
8  import VolunteerFormPage from './pages/VolunteerFormPage';
9  import NotFoundPage from './pages/NotFoundPage';
10 import './styles/App.css';
11
12 function App() {
13   return (
14     <Router>
15       <div className="App">
16         <Navbar />
17         <Routes>
18           <Route path="/" element={<HomePage />} />
19           <Route path="/about" element={<AboutPage />} />
20           <Route path="/volunteer-list" element={<VolunteerListPage />} />
21           <Route path="/volunteer-form" element={<VolunteerFormPage />} />
22           <Route path="*" element={<NotFoundPage />} />
23         </Routes>
24         <Footer />
25       </div>
26     </Router>
27   );
28 }
29 export default App;
  
```

2. Dockerfile content.

```

# Stage 1: Build the React application for production
# We use a Node.js image to have access to npm and build tools.
FROM node:18-alpine AS builder

# Set the working directory inside the container
WORKDIR /app

# Copy the package.json and package-lock.json files
COPY ./package.json ./
COPY ./package-lock.json ./

# Install the dependencies
RUN npm install
  
```

```

# Copy the rest of the application source code
COPY . .

# Build the app for production
# This command is specified in your package.json scripts
RUN npm run build

# Stage 2: Serve the production build with NGINX
# We use a very lightweight NGINX image to serve the static files.
FROM nginx:stable-alpine

# Copy the built files from the 'builder' stage to NGINX's public
directory
# The 'build' directory is where 'npm run build' outputs the static files
COPY --from=builder /app/build /usr/share/nginx/html

# Expose port 80 to the outside world
EXPOSE 80

# The default NGINX command runs, which serves the content
CMD ["nginx", "-g", "daemon off;"]

```

3. Commands with output

a. Create image

Terminal

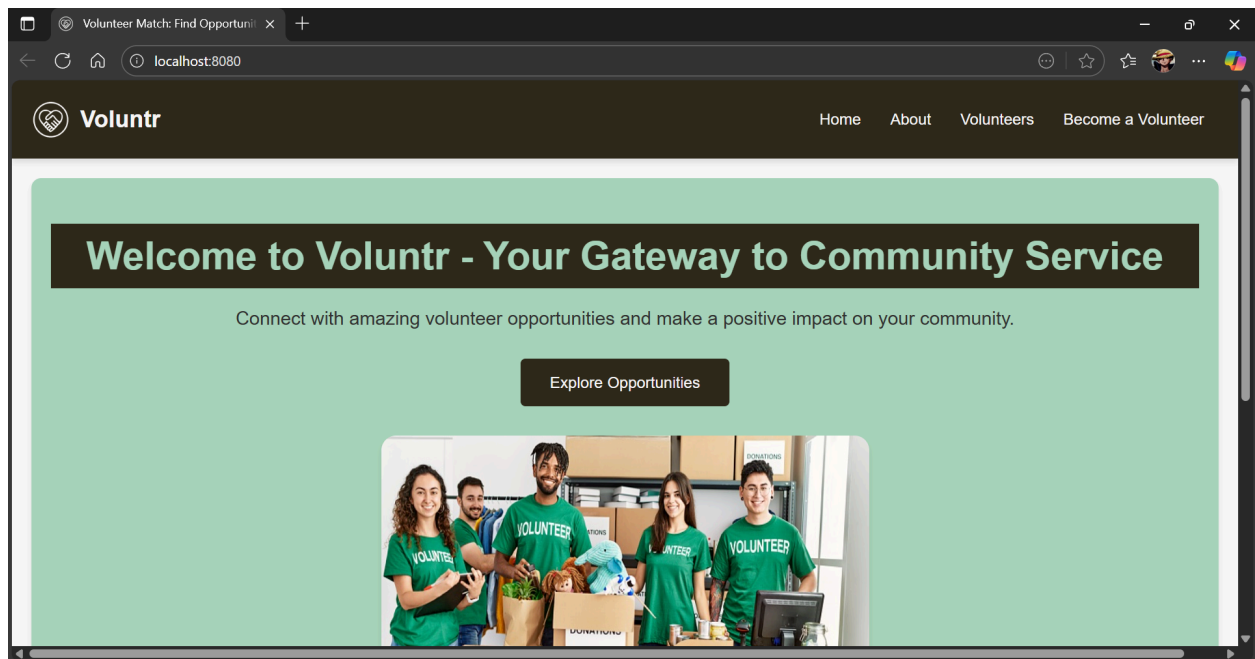
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! <https://aka.ms/PSWindows>

PS C:\Users\Dell> cd C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website
PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> docker build -t chandanagalgal/voluntr-app:1.0 .
[+] Building 215.6s (15/15) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 1.08kB 0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine 1.8s
=> [internal] load metadata for docker.io/library/nginx:stable-alpine 1.8s
=> [internal] load .dockerignore 0.1s

Terminal

=> => transferring context: 149B 0.0s
=> [builder 1/7] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d 33.4s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588 0.1s
=> => sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 446B / 446B 0.6s
=> => sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 1.26MB / 1.26MB 1.5s
=> => sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feea161b2b5a3d0e 40.01MB / 40.01MB 28.9s
=> => sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 3.64MB / 3.64MB 4.9s
=> => extracting sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870 0.8s
=> => extracting sha256:dd71dde834b5c203d162902e6b8994cb2309ae049a0eabc4feea161b2b5a3d0e 3.3s
=> => extracting sha256:1e5a4c89cee5c0826c540ab06d4b6b491c96eda01837f430bd47f0d26702d6e3 0.1s
=> => extracting sha256:25ff2da83641908f65c3a74d80409d6b1b62ccfaab220b9ea70b80df5a2e0549 0.1s
=> [internal] load build context 5.0s
=> => transferring context: 18.87MB 4.7s



d. Push the image on Docker Hub.

```
Terminal
PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> docker push chandanagali/voluntr-app:1.0
The push refers to repository [docker.io/chandanagali/voluntr-app]
2b2bd58f75f4: Pushed
f2d0d37ae31d: Pushed
669998b1fdf4: Pushed
62c31b33012d: Pushed
e585282f4286: Pushed
85a0985bea3a: Pushed
2744e60d29aa: Pushed
0368fd46e3c6: Pushed
8485f215491f: Pushed
95be0f4718a4: Pushed
1.0: digest: sha256:a3e73fffe0badb72f715fc2e7c1b6b18e0279c871bcfb66a96b163892c8f05f6 size: 856
```

e. Deployment code for Kubernetes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: voluntr-app-deployment
  labels:
    app: voluntr-app
spec:
  replicas: 2 # You can change this to 1 or more to control how many
instances run.
  selector:
    matchLabels:
      app: voluntr-app
  template:
    metadata:
      labels:
        app: voluntr-app
    spec:
```

```

containers:
  - name: voluntr-app-container
    image: chandanagalgal/voluntr-app:1.0 # This is the image you
built and pushed.
    ports:
      - containerPort: 80 # NGINX serves the website on this port.

```

```

apiVersion: v1
kind: Service
metadata:
  name: voluntr-app-service
spec:
  type: LoadBalancer # Exposes the service on a cloud provider's load
balancer.
  selector:
    app: voluntr-app
  ports:
    - protocol: TCP
      port: 80 # The port the service will be exposed on.
      targetPort: 80 # The port the container is listening on.

```

Terminal

```

PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> kubectl apply -f deployment.yaml
deployment.apps/voluntr-app-deployment created
PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> kubectl apply -f service.yaml
service/voluntr-app-service created
PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website>

```

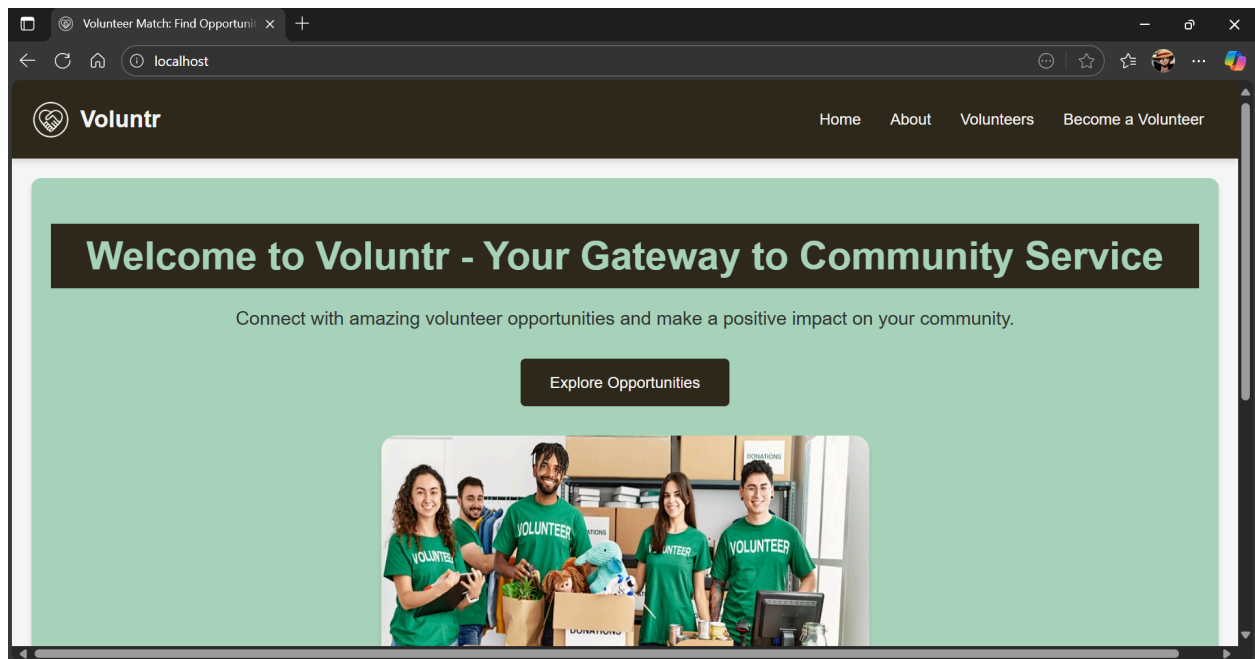
☐		k8s_voluntr-app 88e8a407b4af	a3e73fffe0	0%	7.12MB /		:	
☐		k8s_voluntr-app aeb8bd720029	a3e73fffe0	0%	7.22MB /		:	

Terminal

```

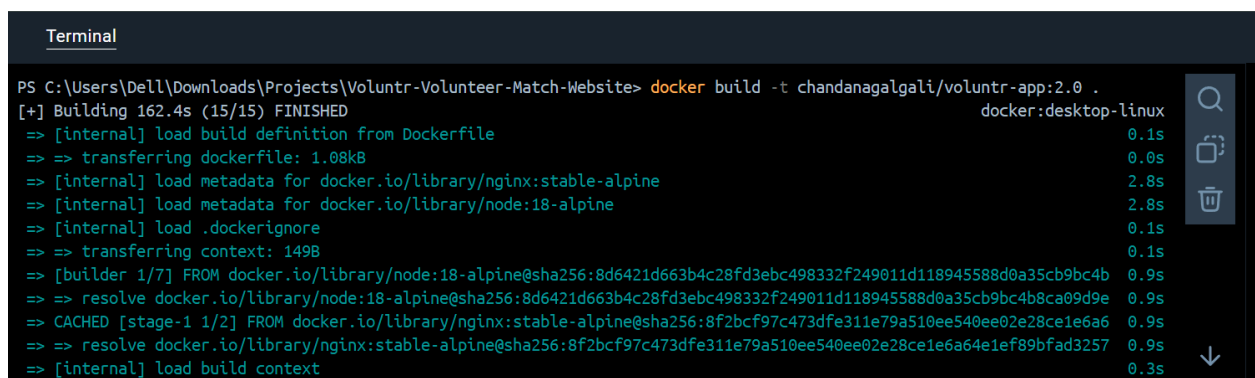
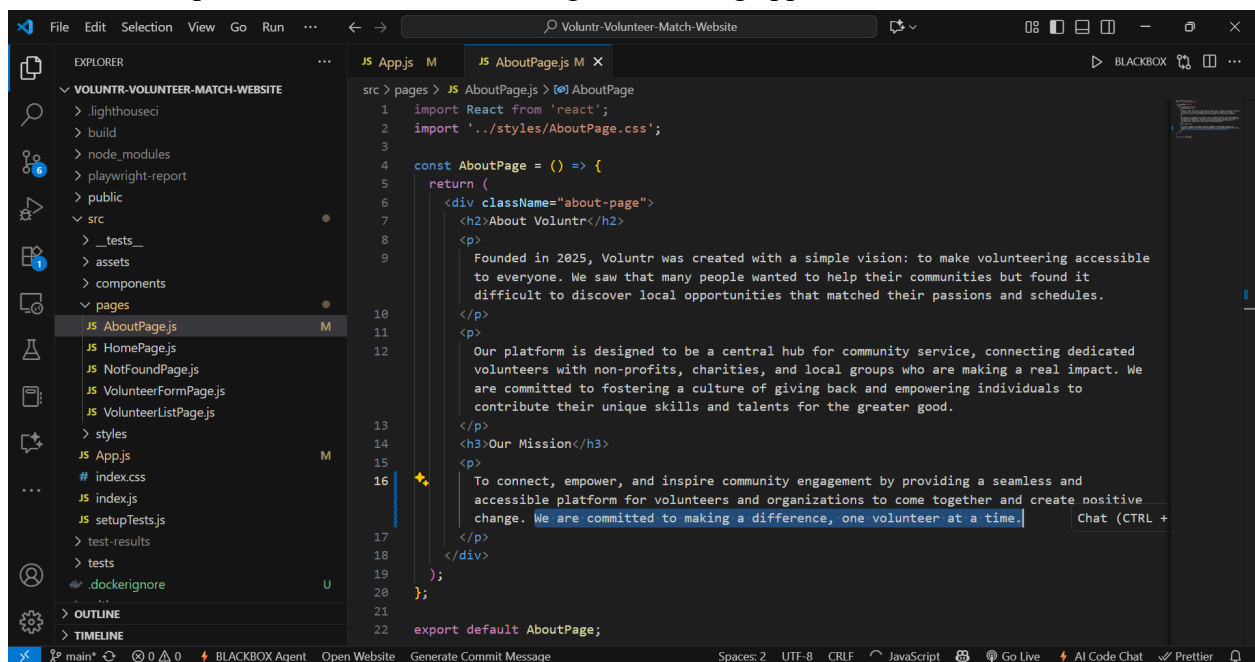
PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
voluntr-app-deployment-8874667dd-hc5gx  1/1     Running   0           36m
voluntr-app-deployment-8874667dd-mzbzf  1/1     Running   0           36m
PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> kubectl get services
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
kubernetes      ClusterIP   10.96.0.1     <none>       443/TCP    39m
voluntr-app-service  LoadBalancer 10.103.212.112 localhost    80:30702/TCP 36m

```

f. Running application

4. Push Updated code and see the changes in running application



Terminal

```

=> => transferring context: 3.77kB
=> CACHED [builder 2/7] WORKDIR /app
=> CACHED [builder 3/7] COPY ./package.json ./
=> CACHED [builder 4/7] COPY ./package-lock.json ./
=> CACHED [builder 5/7] RUN npm install
=> CACHED [builder 6/7] COPY . .
=> [builder 7/7] RUN npm run build
=> [stage-1 2/2] COPY --from=builder /app/build /usr/share/nginx/html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:5fa25566fec2a2593afd1f826a1cfbe604a4293fdeb55cfd42dd4ce3d1a97d
=> => exporting config sha256:2ba6196a1ba01cf3ab7fb904c4484dc87ab94e21a0d8f6cd922bf6d9ae973d12
=> => exporting attestation manifest sha256:a240df587ea8b40ee3df9069939ac5f69542326e6d7b67c061b11af4f7b201b9

```

Terminal

```

=> CACHED [builder 5/7] RUN npm install
=> CACHED [builder 6/7] COPY . .
=> [builder 7/7] RUN npm run build
=> [stage-1 2/2] COPY --from=builder /app/build /usr/share/nginx/html
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:5fa25566fec2a2593afd1f826a1cfbe604a4293fdeb55cfd42dd4ce3d1a97d
=> => exporting config sha256:2ba6196a1ba01cf3ab7fb904c4484dc87ab94e21a0d8f6cd922bf6d9ae973d12
=> => exporting attestation manifest sha256:a240df587ea8b40ee3df9069939ac5f69542326e6d7b67c061b11af4f7b201b9
=> => exporting manifest list sha256:8e770c51a5bf180326a833bce75a969171b3f1acaec2fb2e1b4fdc727a5f1ce6
=> => naming to docker.io/chandanagalgal/voluntr-app:2.0
=> => unpacking to docker.io/chandanagalgal/voluntr-app:2.0

```

PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website>

Terminal

```

PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> docker push chandanagalgal/voluntr-app:2.0
The push refers to repository [docker.io/chandanagalgal/voluntr-app]
a4a79e630be8: Pushed
85a0985bea3a: Layer already exists
669998b1fdf4: Layer already exists
95be0f4718a4: Layer already exists
62c31b33012d: Layer already exists
0368fd46e3c6: Layer already exists
8485f215491f: Layer already exists
f2d0d37ae31d: Layer already exists
e585282f4286: Layer already exists
50c0dda6190b: Pushed
2.0: digest: sha256:8e770c51a5bf180326a833bce75a969171b3f1acaec2fb2e1b4fdc727a5f1ce6 size: 856

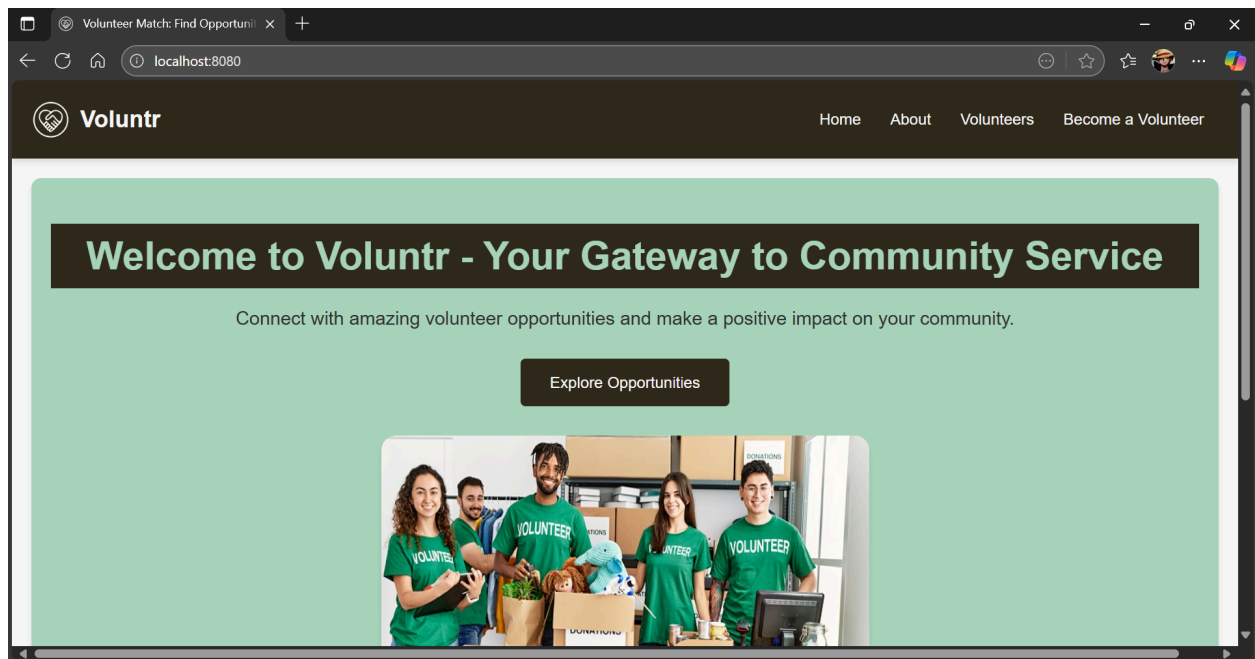
```

```

PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> kubectl apply -f deployment.yaml
deployment.apps/voluntr-app-deployment configured
PS C:\Users\Dell\Downloads\Projects\Voluntr-Volunteer-Match-Website> kubectl port-forward deployment/voluntr-app-deployment 80
80:80
Forwarding from 127.0.0.1:8080 -> 80

```

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Disk read/write	Network I/O	PIDS	Last started	Actions
☐	my-voluntr-app	1f2f36c51aee	chandanagalgal/voln	8080:80	0%	10.71MB / 2.85Gi	0.37%	0B / 0B	34KB / 15.2KB	9	3 hours ago	
☐	k8s_voluntr-app-cont	7c0bccc0e2ab	8e770c51a5bf		0%	8.22MB / 2.85GB	0.28%	0B / 0B	0B / 0B	9	3 minutes ago	
☐	k8s_voluntr-app-cont	bb3ec0da034f	8e770c51a5bf		0%	6.97MB / 2.85GB	0.24%	0B / 0B	0B / 0B	9	2 minutes ago	



Questions:

1. What is Docker file and Docker compose?

Ans: A Dockerfile is a text file that contains a set of instructions used to build a Docker image. Think of it as a blueprint for creating an image. These instructions specify everything needed to create a running container, including the base operating system, application code, dependencies, environment variables, and commands to execute when the container starts. Each instruction in a Dockerfile creates a layer in the Docker image, promoting efficiency and reusability. Docker Compose is a tool that allows you to define and run multi-container Docker applications. Instead of managing individual containers with separate **docker run** commands, Docker Compose uses a YAML file (typically **docker-compose.yml**) to configure all the services of your application. This file describes the services, networks, and volumes needed for your application, making it easy to start, stop, and rebuild the entire application stack with a single command. It simplifies the orchestration of complex applications composed of several interconnected services (e.g., a web application with a database and a caching service).

2. Explain two containers other than Docker.

Ans: While Docker is the most widely known and used containerization technology, several other container runtimes and platforms offer similar functionalities. Here are two prominent examples:

1) Containerd:

- What it is: Containerd is an industry-standard core container runtime that manages the complete container lifecycle of a system. It's a daemon that manages container creation, execution, supervision, and destruction. Docker itself uses containerd under the hood to manage its containers.
- Key Features: It provides a client (CLI tool **ctr**) to interact with it, and handles

image push/pull, storage, and network attachment for containers. Containerd focuses on simplicity, robustness, and portability. It's designed to be embedded into larger systems like Kubernetes.

- Analogy: If Docker is like a complete car that you can drive, containerd is the engine that makes the car move. It's a foundational component.

II) Podman:

- What it is: Podman (POD MANager) is a daemonless container engine for developing, managing, and running OCI (Open Container Initiative) containers on a Linux system. It's often seen as a direct alternative to Docker, offering a very similar command-line interface.
- Key Features: A significant difference from Docker is that Podman does not require a daemon to run, which can improve security. It can run containers as rootless, meaning they don't need elevated privileges. Podman is also compatible with Docker images and supports the same image formats. It's designed to be a drop-in replacement for Docker for many use cases, particularly in development and single-host deployments.
- Analogy: If Docker is a popular brand of a modular building toy set, Podman is another brand that uses the same interlocking pieces and offers a very similar building experience, sometimes with different benefits like not needing a central "master builder" (daemon).

Outcomes: CO4 – Explain code deployment and monitoring systems and their tool support.

Conclusion: (Conclusion to be based on the Results and outcomes achieved)

This experiment successfully demonstrated the end-to-end process of containerizing a NodeJS application using Docker. We began by creating a Dockerfile to define the build instructions, then used it to construct a Docker image. This image was subsequently run as a Docker container, verifying the application's functionality within an isolated environment. Finally, the image was pushed to Docker Hub for easy distribution and version control. The deployment of the Docker image to a Kubernetes cluster using a deployment and service further showcased how containerized applications can be orchestrated for scalability and management, and how updates to the code could be seamlessly deployed. This comprehensive procedure highlights the benefits of Docker and Kubernetes in streamlining application development, deployment, and management workflows.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

References:

Books/ Journals/ Websites:

1. <https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-18-04>
 2. <https://www.simplilearn.com/tutorials/docker-tutorial/install-docker-on-windows>
 3. <https://www.simplilearn.com/tutorials/docker-tutorial/how-to-install-docker-on-ubuntu>
 4. <https://www.whitesourcesoftware.com/free-developer-tools/blog/docker-images-vs-docker-containers/>
 5. Docker Commands Tutorial: <https://www.edureka.co/blog/docker-commands/>
 6. Docker Basic Commands | Docker Commands with Examples | Docker Commands Tutorial | Intellipaat: <https://www.youtube.com/watch?v=nXV6qihj5uw>
 7. <https://www.simplilearn.com/tutorials/docker-tutorial/what-is-dockerfile>
-